

PYTHON FROM ZERO

SESSION 03

Programming Logic & Problem Solving

*From a problem → to a decision → to working Python***What this session is about**

You already know how to give Python instructions using input, variables, calculations and print(). Today, you learn how to make a program decide what to do.

SESSION 3 AT A GLANCE	
Main idea	Programs can make decisions.
Core tools	Boolean values, comparisons, if, else, elif
Main payoff	Implement the voting algorithm from Session 1.
Next step	Prepare the logic for the Student Result Manager in Session 4.1.

1. Start with the problem

In Session 1, you described the steps for a voting-eligibility program without writing Python code. Today, we return to that same logic and give Python the tools needed to execute it.

YOUR SESSION 1 LOGIC	
1	Ask for the person's age.
2	Receive the age.
3	Check the age against the eligibility requirement.
4	If the person is eligible, report it.
5	Otherwise, report that the person is not eligible.

The important question

How does Python actually make the decision at step 3?

A program sometimes needs to choose between different actions. The choice depends on a condition.

REAL-LIFE QUESTION	POSSIBLE ACTION
Is it raining?	Take an umbrella.
Is the password correct?	Allow access.
Is the score high enough?	Report a pass.
Is the person old enough?	Report eligibility.

The computer needs a clear condition that can be evaluated as either True or False.

2. Boolean: the computer's True or False

A Boolean value has only two possible answers: True or False. Think of it as a computer-readable yes/no answer.

QUESTION	ANSWER	PYTHON BOOLEAN
Is 5 greater than 2?	Yes	True
Is 2 greater than 5?	No	False
Is 10 equal to 10?	Yes	True
Is 7 equal to 9?	No	False

Let Python answer

```
print(5 > 2)
print(2 > 5)
print(10 == 10)
print(7 == 9)
```

OUTPUT
True
False
True
False

The comparison produces a Boolean answer. Another part of the program can use that answer to decide what to do.

Boolean values from variables

```
age = 20
print(age >= 18)

age = 15
print(age >= 18)
```

Remember

A comparison asks a question. True or False is the answer.

3. From True/False to a decision

Python uses `if` to tell the computer what to do when a condition is True.

WORD	MEANING
<code>if</code>	Do this when the condition is True.
condition	The question Python evaluates.
indented code	The action controlled by the <code>if</code> .

```
age = 20

if age >= 18:
    print("Eligible")
```

Read it as: "If age is greater than or equal to 18, print 'Eligible'."

What if the condition is False?

```
age = 15

if age >= 18:
    print("Eligible")
```

Nothing is printed because the condition is False. We need another path for the opposite result.

Indentation matters

The indented line belongs to the `if` statement. In Python, indentation is part of the syntax.

Quick check

Predict the output before running this:

```
score = 80

if score >= 50:
    print("Pass")
```

Answer: Pass

4. else: the other path

else tells Python what to do when the if condition is False.

```
age = 15

if age >= 18:
    print("Eligible")
else:
    print("Not eligible")
```

CONDITION	RESULT
age >= 18 is True	print Eligible
age >= 18 is False	print Not eligible

Connect it to input()

```
age = int(input("Enter your age: "))

if age >= 18:
    print("You are eligible to vote.")
else:
    print("You are not eligible to vote.")
```

Session 2 gave you input(), variables, numbers and print(). Session 3 adds the decision-making tools.

SESSION 2	SESSION 3
input()	comparison
int()	Boolean True/False
variables	if / else
print()	decision

This is the click

You already knew how to collect data and work with values. Now Python can use those values to choose an action.

5. The Session 1 algorithm becomes Python

This is the problem you solved in Session 1. Today, you have enough Python knowledge to make the computer execute it.

ALGORITHM	PYTHON
Ask for age	<code>input()</code>
Receive the age	store it in <code>age</code>
Check eligibility	<code>age >= 18</code>
If eligible	<code>if</code>
Otherwise	<code>else</code>
Report the result	<code>print()</code>

Complete program

```
age = int(input("Enter your age: "))

if age >= 18:
    print("You are eligible to vote.")
else:
    print("You are not eligible to vote.")
```

Dry run it before you trust it

TEST AGE	<code>age >= 18</code>	EXPECTED OUTPUT
17	False	Not eligible
18	True	Eligible
20	True	Eligible

Testing both sides of a decision is important. Test the boundary value 18 as well as values below and above it.

6. elif: more than two possibilities

Some problems have more than two possible outcomes. `elif` lets Python check another condition when the earlier `if` condition was `False`.

```
score = float(input("Enter your score: "))

if score >= 70:
    print("Excellent")
elif score >= 50:
    print("Passed")
else:
    print("Failed")
```

SCORE	PATH
70 or more	if
50 to 69.99	elif
Below 50	else

Python checks conditions from top to bottom. Once it finds a `True` condition, it runs that block and skips the remaining alternatives.

Do not rush this

First understand the simple pattern: `if` → otherwise `else`. Add `elif` when a real problem needs another path.

Quick prediction

```
score = 45

if score >= 70:
    print("Excellent")
elif score >= 50:
    print("Passed")
else:
    print("Failed")
```

Output: Failed

7. A simple problem-solving method

Good programming is not just typing code quickly. Move from the problem to a tested solution.

STEP	QUESTION TO ASK
1. Understand	What exactly are we trying to solve?
2. Identify inputs	What information does the program need?
3. Identify the process	What calculations or decisions are needed?
4. Identify outputs	What should the user see?
5. Write the steps	What should happen, in order?
6. Code	How do we express those steps in Python?
7. Test	What happens with different inputs?

Example: voting eligibility

Problem → A person wants to know whether they are eligible to vote.

Input → age

Decision → is age greater than or equal to 18?

Output → eligible / not eligible

Code → use `input()`, comparison, `if`, `else` and `print()`

Test → try 17, 18 and 20

A useful habit

Before coding, be able to explain what the program should do in plain language. Code should express the logic—not replace the thinking.

8. Bridge to Session 4: Student Result Manager

Now consider a larger problem. A school wants a small program that can process a student's result.

Do not code it yet. Break it down.

PART	WHAT THE PROGRAM NEEDS TO DO
1. Student	Get the student's name.
2. Scores	Get the required scores.
3. Calculation	Calculate the total.
4. Calculation	Calculate the average.
5. Decision	Determine the result or grade.
6. Output	Display the student's result.

The bigger picture

```

Get student information
↓
Get scores
↓
Calculate total and average
↓
Make a decision
↓
Determine result
↓
Display result

```

What you already know

FROM SESSION 2	FROM SESSION 3
input()	Boolean True / False
variables	comparisons
numbers	if
arithmetic	else / elif
print()	testing decisions

Next session

In Session 4.1, we will combine these pieces to start building the Student Result Manager.

9. Practice & session checklist

Practice 1 — Predict

Predict the output before running each program.

```
print(12 > 5)
print(4 == 9)
```

```
age = 16
print(age >= 18)
```

Practice 2 — Complete the decision

Write the missing part so the program prints “Pass” when the score is 50 or more.

```
score = 72

if ____:
    print("Pass")
```

Practice 3 — Build the logic

Create a program that asks for a person's score and prints:

CONDITION	OUTPUT
70 or more	Excellent
50–69	Passed
Below 50	Failed

Before you leave this session, you should be able to...

- Explain True and False in your own words.
- Read a comparison such as `age >= 18`.
- Explain what `if` does.
- Explain what `else` does.
- Explain when `elif` is useful.
- Turn a simple algorithm into Python.
- Test a program using different inputs.
- Break a larger problem into smaller steps.

The main idea

Programming becomes powerful when your code can respond differently to different situations. Today you learned how to make that happen.